COMP 2154 — System Development Project

Testing Documentation

OzarkLMS

Team OzarkLMS

Luis Tello | Gonzalo Contaldo | Ifrad Bin Istiaque Hossain

John Sebastian Laquis | Sanyoung Yoon

Document Version: 1.0 (Final) | April 2026

1. Testing Overview

1.1 Purpose

This document provides a comprehensive record of the testing strategy, test case specifications, execution results, and defect tracking for OzarkLMS. It serves as the formal testing appendix to the OzarkLMS Final Project Report and demonstrates how system quality was validated across functional and non-functional dimensions.

The document covers:

- Testing strategy and approach across unit, integration, and system levels.
- Full test case specifications for all features implemented in OzarkLMS.
- Requirements Traceability Matrix linking each functional and non-functional requirement to its test cases.
- Actual test execution results, metrics, and defect resolution.
- Known limitations and future testing recommendations.

1.2 Scope

In scope for testing:

- User Authentication (registration, login, logout, role-based access control)
- Course and Module Management (CRUD, enrollment, module items)
- Assignment and Quiz Creation (file attachments, quiz builder, auto-grading)
- Student Submission (file upload, text entry via Quill, multi-file)
- Grading and Feedback (score entry, feedback text, gradebook)
- Notifications (automated alerts, broadcast, targeted delivery)
- Collaboration Features (group chat, private DM, social feed, voting, comments)
- PDF Toolbox (merge, text-to-PDF conversion, document editor export)
- Calendar (personal events CRUD, due-date overlay)
- Admin Dashboard (user management, soft delete)

Out of scope:

- Performance stress/load testing under concurrent users
- Penetration testing or security audits beyond basic authorization checks
- Accessibility/WCAG audit
- Cross-browser automation (manual browser testing only)

1.3 Test Environment

Component	Details
Operating System	Operating System Agnostic (Native to Windows, Mac, iOS, and Android)
Browsers	Google Chrome 134, Microsoft Edge 134, Firefox 136, eg. Chromium Browsers
Runtime	.NET 9.0 / ASP.NET Core 9.0
Database	PostgreSQL 16 (local) / Azure PostgreSQL Flexible Server (cloud)
Unit Test Framework	xUnit 2.x + Moq + EF Core InMemory provider
Test Data	DbInitializer seeds: admin, instructor, student accounts; 3 courses; sample assignments
Deployment URL	https://ozarklmscrakb2dqdqb4crfn.canadacentral-01.azurewebsites.net/Account/Login

2. Test Strategy

2.1 Testing Levels

Level	Objective	Tools	When
Unit Testing	Individual controller actions and service logic	xUnit, Moq, EF Core InMemory	During development
Self-Diagnostic	Runtime health checks (DB connectivity, data integrity)	SelfTestService (custom)	On application startup
Integration Testing	Controller → EF Core → PostgreSQL interaction	xUnit + real PostgreSQL	Feature completion
System / E2E Testing	Complete user workflows across all roles	Manual browser testing	Weeks 10-13

2.2 Testing Principles

- All automated tests follow the AAA pattern (Arrange, Act, Assert).
- Tests are isolated from the production PostgreSQL database using EF Core InMemory.
- Both positive (happy path) and negative (validation/error) scenarios are covered.
- Manual system tests cover all three user roles: Student, Instructor, and Admin.
- Regression testing is performed after every defect fix.

2.3 Requirements Traceability Matrix

Req. ID	Description	Test Cases	Status	Result
FR-1	User Authentication & Roles	TC-AUTH-001, TC-AUTH-002, TC-AUTH-003, TC-AUTH-004	Met	PASS
FR-2	Course Management	TC-COURSE-001, TC-COURSE-002	Met	PASS
FR-3	Content / Modules	TC-MODULE-001	Met	PASS
FR-4	Assignment Creation	TC-ASSIGN-001, TC-ASSIGN-002 (TC-02)	Met	PASS
FR-5	Quiz Management	TC-QUIZ-001, TC-QUIZ-002	Met	PASS
FR-6	Student Submission	TC-SUBMIT-001 (TC-03)	Met	PASS
FR-7	Grading & Feedback	TC-GRADE-001 (TC-04)	Met	PASS
FR-8	Calendar	TC-CAL-001	Met	PASS
FR-9	Notifications	TC-NOTIF-001, TC-NOTIF-002	Met	PASS
FR-10	Collaboration / Chat	TC-COLLAB-001, TC-COLLAB-002	Met	PASS
NFR-1	Performance	TC-NFR-PERF-001	Met	PASS
NFR-2	Security / Auth	TC-AUTH-004, TC-RBAC-001, TC-RBAC-002	Partial	PARTIAL
NFR-3	Usability	TC-NFR-USAB-001	Met	PASS
NFR-4	Reliability	TC-ASSIGN-002, TC-SUBMIT-001	Met	PASS

Req. ID	Description	Test Cases	Status	Result
NFR-5	Compatibility	TC-NFR-COMPAT-001	Met	PASS

Note: NFR-2 (Security) is marked Partial because password hashing has not been implemented. All authentication flow tests pass, but plaintext password storage remains a known gap.

3. Automated Test Cases (xUnit)

The following test cases were implemented in the OzarkLMS.Tests project using xUnit, Moq, and the EF Core InMemory provider. All 8 tests passed with a total run time of approximately 939 ms.

TC-01 / TC-AUTH-001 — Successful Student Registration & Login

Requirement	FR-1 — User Authentication
Priority / Type	High / Positive (Automated — xUnit)
Pre-Conditions	No pre-existing user with the same username; InMemory database initialized; IAuthenticationService mocked.
Steps	1. Initialize InMemory AppDbContext. 2. Mock IAuthenticationService to capture sign-in calls. 3. Instantiate AccountController with mocked dependencies. 4. Call Register POST with a new RegisterViewModel (username, password, role='student'). 5. Assert redirect to Home/Index. 6. Call Login POST with the same credentials. 7. Assert user is authenticated and redirected to Home/Index.
Expected Results	• Registration creates a User record in the InMemory database. • Login succeeds and returns a RedirectToActionResult targeting 'Index'. • No error messages are added to ModelState. • Cookie authentication sign-in is invoked once with correct claims.
Result	**PASS**

TC-02 / TC-ASSIGN-001 — Instructor Creates Assignment

Requirement	FR-4 — Assignment Creation
Priority / Type	High / Positive (Automated — xUnit)
Pre-Conditions	Instructor user exists in InMemory DB; Course with matching InstructorId exists; Controller has instructor identity mocked.
Steps	1. Initialize InMemory AppDbContext with a Course (Id=1, InstructorId=1). 2. Instantiate AssignmentsController with mocked IWebHostEnvironment. 3. Set HttpContext user to instructor (Id=1, role='instructor'). 4. Call Create POST with Assignment (Title='Midterm Project', Points=100, Type='assignment', CourseId=1, DueDate=tomorrow). 5. Assert result is RedirectToActionResult. 6. Query InMemory DB for the new assignment.
Expected Results	• Result is a RedirectToActionResult targeting 'Details' in 'Courses'. • An Assignment record with Title='Midterm Project' and Points=100 exists in the database. • Assignment.CourseId matches the provided CourseId. • Enrolled students receive a notification record.
Result	**PASS**

TC-03 / TC-SUBMIT-001 — Student Submits File Assignment

Requirement	FR-6 — Student Submission
Priority / Type	High / Positive (Automated — xUnit)
Pre-Conditions	Student user (Id=2) exists; Assignment (Id=1) exists; IWebHostEnvironment.WebRootPath mocked to temp directory.
Steps	1. Initialize InMemory DB with student and assignment records. 2. Mock IWebHostEnvironment to return a temp path. 3. Set HttpContext user to student (Id=2, role='student'). 4. Call SubmitAssignment POST with assignmentId=1, content='My submission', and a mocked IFormFile. 5. Assert result type. 6. Query DB for Submission record.
Expected Results	• Result is a RedirectToActionResult (to Take action with ShowSubmissionBanner in TempData). • A Submission record is created with AssignmentId=1 and StudentId=2. • Submission.Content matches provided text. • If file provided, SubmissionAttachment record is created with correct FileName and FilePath.
Result	**PASS**

TC-04 / TC-GRADE-001 — Instructor Grades Submission

Requirement	FR-7 — Grading & Feedback
Priority / Type	High / Positive (Automated — xUnit)
Pre-Conditions	Submission (Id=1) exists with no score; Instructor user with matching role mocked.
Steps	1. Initialize InMemory DB with a Submission record (Score=null, Feedback=null). 2. Set HttpContext user to instructor. 3. Call GradeSubmission POST with submissionId=1, score=85, feedback='Good work'. 4. Assert result type. 5. Re-query submission from DB.
Expected Results	• Result is a RedirectToActionResult targeting 'Submissions'. • Submission.Score is updated to 85. • Submission.Feedback is updated to 'Good work'. • No exceptions thrown.
Result	**PASS**

TC-06 / TC-PROFILE-001 — User Updates Bio

Requirement	FR-2 — Profile Management
Priority / Type	Medium / Positive (Automated — xUnit)
Pre-Conditions	User (Id=1) exists in InMemory DB with Bio=null; AccountController instantiated with InMemory context.
Steps	1. Initialize InMemory DB with User (Id=1, Username='student1', Bio=null). 2. Set HttpContext user to student (Id=1). 3. Call UpdateBio POST with bio='Hello world, I am a student.'. 4. Assert result type. 5. Re-query user from DB.

Expected Results	• Result is a RedirectToActionResult targeting 'Profile'. • User.Bio in database is updated to 'Hello world, I am a student.'. • No ModelState errors.
Result	**PASS**

4. Self-Diagnostic Test Cases (SelfTestService)

The SelfTestService runs automatically when navigating to /Test/Index and validates runtime health. These tests execute against the live database.

SELF-01 — Math Logic Unit Test

Requirement	Internal Quality — SelfTestService
Priority / Type	Low / Positive (Runtime)
Pre-Conditions	SelfTestService is instantiated with AppDbContext.
Steps	1. Call TestMathLogic() method. 2. Compute 5 + 10 = 15. 3. Assert result equals 15.
Expected Results	• TestResult.Passed = true. • TestResult.Message = '5 + 10 correctly equals 15.'
Result	**PASS**

SELF-02 — Database Connectivity

Requirement	NFR-4 — Reliability; NFR-2 — Security
Priority / Type	High / Positive (Runtime)
Pre-Conditions	PostgreSQL database (OZARK_DB) is running and connection string is correctly configured.
Steps	1. Call TestDatabaseConnectionAsync(). 2. Invoke _context.Database.CanConnectAsync(). 3. Assert the result.
Expected Results	• TestResult.Passed = true. • TestResult.Message = 'Successfully connected to the database.'
Result	**PASS**

SELF-03 — User Data Verification

Requirement	Internal Quality — SelfTestService
Priority / Type	Medium / Positive (Runtime)
Pre-Conditions	DbInitializer has seeded at least one user (admin, instructor, student).
Steps	1. Call TestUserCountAsync(). 2. Count users in _context.Users. 3. Assert count > 0.
Expected Results	• TestResult.Passed = true. • TestResult.Message contains the user count.
Result	**PASS**

SELF-04 — Course Data Verification

Requirement	Internal Quality — SelfTestService
Priority / Type	Medium / Positive (Runtime)
Pre-Conditions	DbInitializer has seeded at least one course.
Steps	1. Call TestCourseDataAsync(). 2. Retrieve first course from _context.Courses. 3. Assert non-null.
Expected Results	• TestResult.Passed = true. • TestResult.Message contains course name and code.
Result	**PASS**

5. System / Manual Test Cases

5.1 Authentication Module

TC-AUTH-002 — Unsuccessful Login — Invalid Credentials

Requirement	FR-1 — User Authentication
Priority / Type	High / Negative (Manual)
Pre-Conditions	System is online; no user exists with username 'wrong_user'.
Steps	1. Navigate to /Account/Login. 2. Enter username: 'wrong_user', password: 'badpass'. 3. Click Sign In.
Expected Results	• Login fails; user remains on /Account/Login. • Error message 'Invalid login attempt.' is displayed. • No authentication cookie is set. • No redirect to Dashboard occurs.
Result	**PASS**

TC-AUTH-003 — Registration with Duplicate Username

Requirement	FR-1 — User Authentication
Priority / Type	High / Negative (Manual)
Pre-Conditions	A user with username 'student' already exists in the database.
Steps	1. Navigate to /Account/Register. 2. Fill all fields using username 'student'. 3. Submit the form.
Expected Results	• Registration fails. • Error message 'Username already taken.' is displayed. • User remains on /Account/Register. • No duplicate user record is created.
Result	**PASS**

TC-AUTH-004 — Role-Based Access — Student Cannot Access Admin Panel

Requirement	NFR-2 — Security; FR-1 — Authentication
Priority / Type	High / Negative (Manual)
Pre-Conditions	Logged in as a student role user.
Steps	1. Authenticate as 'student' / 'password'. 2. Attempt to navigate directly to /Admin/Dashboard. 3. Observe the result.
Expected Results	• Access is denied. • User is redirected to /Account/AccessDenied or receives an HTTP 403 response. • Admin dashboard content is not visible.
Result	**PASS**

5.2 Course and Module Management

TC-COURSE-001 — Instructor Creates a New Course

Requirement	FR-2 — Course Management
Priority / Type	High / Positive (Manual)
Pre-Conditions	Logged in as instructor role user.
Steps	1. Navigate to /Courses/Index. 2. Click 'New Course'. 3. Enter Name, Code, Term fields. 4. Click Create.
Expected Results	• Course is created and persisted to the database. • User is redirected to /Courses/Index. • New course appears in the course list. • Course is visible in the instructor's dashboard.
Result	**PASS**

TC-ASSIGN-002 — Create Assignment with Missing Title (Validation)

Requirement	FR-4 — Assignment Validation; NFR-4 — Reliability
Priority / Type	Medium / Negative (Manual)
Pre-Conditions	Logged in as instructor; Course exists.
Steps	1. Navigate to Assignments/Create?courseId=1. 2. Leave the Title field blank. 3. Fill all other required fields (Points, DueDate, Type). 4. Click Create Assignment.
Expected Results	• Form submission is blocked by model validation. • Error message 'The Title field is required.' is displayed below the Title input. • Previously entered values remain populated. • No Assignment record is created in the database. • User remains on the Create Assignment page.
Result	**PASS**

5.3 Quiz Management

TC-QUIZ-001 — Instructor Creates Quiz and Adds Questions

Requirement	FR-5 — Quiz Management
Priority / Type	High / Positive (Manual)
Pre-Conditions	Logged in as instructor; Course exists.
Steps	1. Navigate to Assignments/Create?courseId=1&type=quiz. 2. Enter quiz title and due date, click Create.

	3. On the Edit Quiz page, add a question with 4 options marking one as correct. 4. Click Save Question.
Expected Results	• Quiz Assignment record is created with Type='quiz'. • Question record is created and linked to the Assignment. • QuestionOption records are created; one has IsCorrect=true. • Edit Quiz page displays the question and options.
Result	**PASS**

TC-QUIZ-002 — Student Takes Quiz — Auto-Grading

Requirement	FR-5 — Quiz Management; FR-7 — Grading
Priority / Type	High / Positive (Manual)
Pre-Conditions	Quiz with questions exists; student is enrolled in the course.
Steps	1. Log in as student and navigate to the quiz Take page. 2. Select answers for all questions. 3. Click Submit Quiz.
Expected Results	• Submission record is created with auto-computed Score. • Score reflects correctly answered questions multiplied by their point value. • Student is redirected to Course Details (quizzes tab). • Submission is visible under the Submissions view for the instructor.
Result	**PASS**

5.4 Collaboration and Social Hub

TC-COLLAB-001 — User Creates Post and Receives Real-Time Vote Update

Requirement	FR-10 — Collaboration
Priority / Type	High / Positive (Manual)
Pre-Conditions	Two browser sessions open (User A and User B, both logged in); SignalR connection active.
Steps	1. User A navigates to /Collaboration and creates a new post. 2. User B observes the feed (may require follow relationship for post visibility). 3. User B clicks the upvote button on a post. 4. Observe User A's browser without refresh.
Expected Results	• Post is created and appears in the feed. • User B's upvote triggers a real-time SignalR broadcast via VoteHub. • User A's browser displays the updated vote count without a page refresh. • Vote score and icon color update instantly on all connected clients. • Submission is persisted to the PostVotes table.
Result	**PASS**

TC-COLLAB-002 — Private Chat Message Delivery

Requirement	FR-10 — Collaboration
Priority / Type	High / Positive (Manual)
Pre-Conditions	Two users have an existing private chat session.
Steps	1. Navigate to /Collaboration/PrivateDetails/{chatId} as User A. 2. Type a message and press Enter or click Send. 3. Open the same chat as User B.
Expected Results	• PrivateMessage record is created in the database. • Recipient (User B) receives a Notification record. • Message appears in the chat thread on page refresh. • LastActivityDate on PrivateChat is updated.
Result	**PASS**

5.5 Non-Functional Tests

TC-NFR-PERF-001 — Dashboard Load Performance

Requirement	NFR-1 — Performance
Priority / Type	Medium / Positive (Manual)
Pre-Conditions	Application running locally; student enrolled in 3 courses with assignments.
Steps	1. Open browser DevTools (Network tab). 2. Log in as student and navigate to /Home/Index. 3. Record total page load time.
Expected Results	• Dashboard fully renders within 2 seconds under normal local load. • No blocking requests or errors in the browser console. • All course cards, sticky notes, and calendar widget render correctly.
Result	**PASS**

TC-NFR-COMPAT-001 — Cross-Browser Compatibility

Requirement	NFR-5 — Compatibility
Priority / Type	Medium / Positive (Manual)
Pre-Conditions	Application running on Azure; latest versions of Chrome, Edge, Firefox.
Steps	1. Open the application on Chrome 134. 2. Perform a login, navigate to Courses, create/view an assignment. 3. Repeat on Microsoft Edge 134. 4. Repeat on Firefox 136.
Expected Results	• All core UI components (sidebar, dark mode toggle, modals, forms) render correctly in all three browsers. • Tailwind CSS styles apply consistently. • SignalR connection establishes successfully in all browsers. • No JavaScript console errors related to browser incompatibility.
Result	**PASS**

6. Test Execution Summary

6.1 Overall Metrics

Category	Total	Passed	Failed	Skipped	Pass Rate
Authentication	5	5	0	0	100%
Course / Module Mgmt	3	3	0	0	100%
Assignments & Quizzes	4	4	0	0	100%
Submission & Grading	2	2	0	0	100%
Collaboration / Social	2	2	0	0	100%
Self-Diagnostic	4	4	0	0	100%
Non-Functional	2	2	0	0	100%
TOTAL	**22**	**22**	**0**	**0**	**100%**

Automated xUnit Run:

Passed: 8 | Failed: 0 | Skipped: 0 | Total: 8 | Duration: ~939 ms

Command used to run automated tests:

```
dotnet test OzarkLMS.Tests/OzarkLMS.Tests.csproj --verbosity normal
```

6.2 Automated Test Execution Result

Test run output (condensed):

```
Passed!  - Failed: 0, Passed: 8, Skipped: 0, Total: 8, Duration: 939 ms - OzarkLMS.Tests.dll
(net9.0)
```

7. Defect Tracking

ID	Description	Severity	Status	Resolution
DEF-001	Duplicate SignalR connection.start() call in Collaboration/Index caused console errors	Medium	Resolved	Added guard: if (typeof connection !== 'undefined') before second start() call
DEF-002	DateTime fields without UTC kind caused EF Core InMemory test failures	Medium	Resolved	Added DateTime.SpecifyKind(..., DateTimeKind.Utc) on all date assignments
DEF-003	Soft-delete filter excluded needed users in admin duplicate-username check	Low	Resolved	Added IgnoreQueryFilters() explicitly in AccountController and AdminController username checks
DEF-004	DOCX-to-PDF conversion strips formatting (tables, images, headings)	Low	Known Limitation	Documented; PDFToolboxController.ConvertDocxToPDF uses Body.InnerText only; full formatting conversion deferred to future work

8. Known Limitations and Future Testing Recommendations

8.1 Current Limitations

- Passwords stored in plaintext: The most critical security gap. All authentication tests pass the current implementation but production deployment requires bcrypt or ASP.NET Core Identity hashing before any live use.
- File storage is local: Uploaded files are stored in wwwroot/uploads/ which is not horizontally scalable. No automated test validates cloud storage integration.
- SignalR broadcasts globally: VoteHub does not scope messages to room participants. Testing confirmed correct delivery but excessive data is sent to all connected clients.
- No automated E2E tests: All system-level test cases were executed manually. Automation with Playwright or Selenium is recommended for regression coverage.

8.2 Future Testing Recommendations

- Implement password hashing (bcrypt) and add a test case verifying hashed storage.
- Add Playwright or Selenium automated E2E test suite covering the full student and instructor workflows.
- Conduct load testing with a tool such as Apache JMeter to verify NFR-1 performance under 50+ concurrent users.
- Implement SignalR group-scoped messaging and add test cases verifying message isolation between chat rooms.
- Add DOCX-to-PDF conversion tests once full formatting preservation is implemented.
- Integrate the xUnit test suite into a GitHub Actions CI/CD pipeline for automated test runs on every pull request.

End of Testing Documentation — OzarkLMS | COMP 2154 | April 2026